

Tecnológico de Costa Rica

Proyecto #3

Curso:

Taller de Programación

Profesor:

Cristian Paz Campos Aguero

Estudiantes:

Deykell Reyes Campos

Justin Díaz

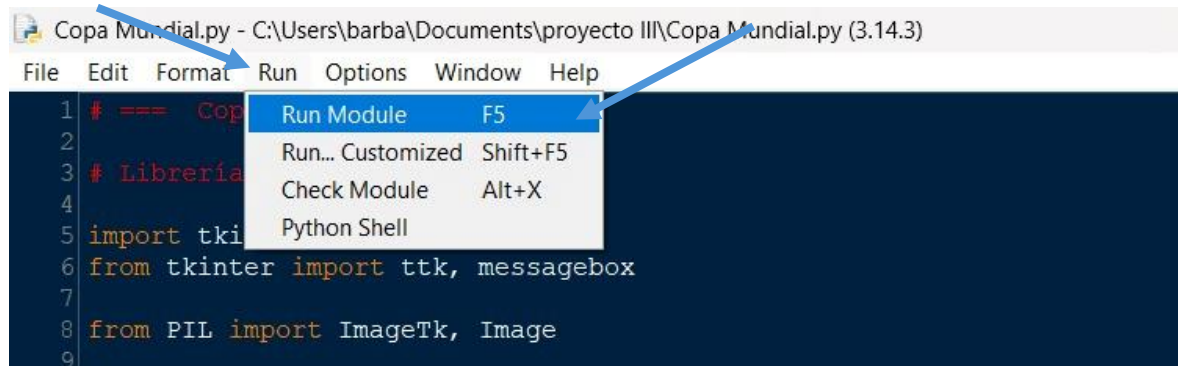
Grupo 61

Año

2026

Manual de usuario: instrucciones de compilación, ejecución y uso.

Para ejecutar el programa, primero se debe tener abierto el código del mismo, si se ejecuta desde el IDLE de Python, se debe presionar en el botón de “Run”, luego la opción que dice “Run Module” y se ejecuta el programa. A continuación, se muestra una imagen de referencia:



Al ejecutar el programa, se abrirá la ventana principal del programa o la ventana del menú de las opciones para realizar. La ventana es llamada “Ventana Principal” y en ella se presentan 5 botones, los cuales dirigen a una ventana asignada para cada uno como sus títulos lo dice. A continuación, se muestra una imagen de la ventana principal:



La ventana cuenta con los siguientes botones:

1: “Administración de países y selecciones”: si se presiona ese botón se abrirá una ventana llamada “Administración Países y Selecciones”, en la cual, como su nombre lo dice, se pueden registrar o modificar los países y selecciones que podrán participar en el mundial. A continuación, se muestra una imagen de la ventana de la administración de los países y selecciones.

Administración Países y Selecciones

Registre y Cree sus Países y Selecciones aquí

Regresar al menú principal

Países

Registrar / Editar Países

Código FIFA:

Nombre del País:

Continente:

Ranking FIFA:

+ Añadir País Guardar Cambio Limpiar

Lista de Países Registrados Total: 48 países

Código FIFA	Nombre	Continente	Ranking fifa
GER	Germany	Europa	1
USA	United States	America	2
ARG	Argentina	America	3
FRA	France	Europa	4
ENG	England	Europa	5
BRA	Brazil	America	6
POR	Portugal	Europa	7
NED	Netherlands	Europa	8
ESP	Spain	Europa	9
BEL	Belgium	Europa	10

Registrar / Editar Selección

Código Selección:

País:

Entrenador:

+ Añadir Selección Guardar Cambio Limpiar

Lista de Selecciones Registradas Total: 48 selecciones

Código	País	Entrenador	Jug.	Fuerza
MEX	Mexico	1 2	11	66
CAN	Canada	Tecnico Canada	12	65
BRA	Brazil	Tecnico Brazil	11	94
USA	United States	Tecnico United States	11	81
GER	Germany	Tecnico Germany	11	92
NED	Netherlands	Tecnico Netherlands	11	92
BEL	Belgium	Tecnico Belgium	11	91
ESP	Spain	Tecnico Spain	11	92
FRA	France	Tecnico France	11	95
ARG	Argentina	Tecnico Argentina	11	95

Al lado izquierdo, como se muestra en la imagen, están las opciones para registrar un país, el cual con jugadores y un entrenador puede conformar una selección de fútbol y participar en el mundial. Para registrar un país se deben ingresar los siguientes datos: el código del país, el nombre del país, el continente al que pertenece el país y su ranking FIFA, el cual puede ser modificado conforme a su desempeño en los juegos, al ya haber ingresado los datos del país, se presiona el botón “Añadir País”, o para modificar el ranking FIFA solo se debe presionar el país a modificar y al cambiar el ranking, se debe presionar el botón “Guardar Cambio”. Además, en la pantalla se muestran las selecciones que ya han sido registradas.

Al lado derecho de la ventana están las opciones de registrar una selección de fútbol, la cual será la que podrá participar en el mundial. Para registrar una selección se deben ingresar los siguientes datos: el código de la selección, el país al cual se le está registrando la selección y su entrenador, y para registrar la selección solo se debe presionar el botón “Añadir Selección” o para modificar una se debe seleccionar de la parte inferior y al hacer el cambio se debe presionar el botón “Guardar Cambios”. Además, en la parte inferior de las opciones de registrar la selección se muestran las selecciones que ya han sido registradas.

2: “Administrar Entrenadores y Jugadores”: si se presiona ese botón, se abrirá una ventana llamada “Administración de Entrenadores y Jugadores” en la cual, como su nombre menciona, se pueden registrar jugadores, entrenadores, y a ambos se les puede seleccionar el país al que van a representar. A continuación, se muestra una imagen de la ventana:

Administración de Entrenadores y Jugadores

Registrar / Editar Entrenador

Datos Personales del Entrenador

Nombre:

Apellidos:

Nacionalidad:

Fecha de Nacimiento:

Datos del Entrenador

Licencia:

Experiencia:

Sistema de Juego:

+ Añadir Entrenador **Guardar Cambios** **Limpiar**

Asignar Entrenador a Selección

Selección:

Entrenador:

+ Asignar Entrenador a Selección

Entrenadores Registrados

Entrenador	Nacionalidad	Licencia	Años	Sistema	Selección
Cef Cef	03/07/1954	UEFA C	3	5-3-2	
Dfue Wefr	03/03/1954	UEFA A	3	4-2-3-1	
Wer Jh	Francia	UEFA B	4	4-4-2	

Eliminar Entrenador **Volver**

Registrar / Editar Jugador

Datos Personales del Jugador

Nombre:

Apellidos:

Nacionalidad:

Fecha de Nacimiento:

Datos del Jugador

Dorsal:

Posición:

Velocidad:

Estrategia:

Dominio del Balón:

Fuerza:

+ Añadir Jugador **Guardar Cambios** **Limpiar**

Asignar Jugador a Selección

Jugador:

Selección:

+ Asignar Jugador a Selección

Ver Jugadores por Selección

Ver Jugadores

Jugadores Registrados

Jugador	Nacionalidad	Dorsal	Posición	Puntaje	Selección
Lionel Messi	Argentina	10	Mediocentro	100	Sin asignar
Pessi Cola	Argentina	10	Lateral Derecho	100	Sin asignar

Eliminar Jugador **Volver**

En la parte superior de la ventana se muestran las opciones para registrar o modificar un entrenador, asignarlo a un país y poder eliminar alguno. Para registrar un entrenador se deben ingresar los siguientes datos: nombre del entrenador, apellido o apellidos del entrenador, la nacionalidad del entrenador, el día, mes y año de nacimiento, la licencia con la que cuenta, los años de experiencia y su sistema de juego, el cual se podrá cambiar cuando desee, y para registrar un entrenador, al ingresar todos los datos necesarios solo se debe presionar el botón “Añadir entrenador” o para modificar el sistema de juego, solo se selecciona el entrenador, se modifica el sistema de juego y se presiona el botón “Guardar Cambios”. Para asignar un entrenador a una selección solo se debe seleccionar el entrenador, la selección y presionar el botón “Asignar Entrenador a Selección” y para borrar un entrenador, solo se debe presionar el que se quiera eliminar y se presiona el botón “Eliminar Entrenador”.

En la parte superior de la ventana se muestran las opciones para registrar o modificar un jugador, asignarlo a un país y poder eliminar alguno. Para registrar un jugador se deben ingresar los siguientes datos: nombre del jugador, apellido o apellidos del jugador, la nacionalidad del jugador, el día, mes y año de nacimiento, el número de dorsal, la posición, la velocidad, la estrategia, el dominio del balón y su fuerza, y para registrar un jugador, al ingresar todos los datos necesarios solo se debe presionar el botón “Añadir entrenador” o para modificar uno, solo se selecciona el jugador, se modifican los datos y se presiona el botón “Guardar Cambios”. Para asignar un jugador a una selección solo se debe seleccionar el jugador, la selección y presionar el botón “Asignar Jugador a Selección” y para borrar un jugador, solo se debe presionar el que se quiera eliminar y se presiona el botón “Eliminar jugador”.

3: “Configurar Mundial Países (Grupos)” : si se presiona ese botón, se abrirá una ventana llamada “Configuración del Mundial” en la cual, es en la ventana donde se crearán los grupos dependiendo de las selecciones registradas y los grupos indicados. Para iniciar un mundial se deben ingresar los grupos, 2 mínimo y 12 máximo, y por cada grupo participan 4 selecciones y a cada grupo se le asigna una letra como identificador. A continuación, se mostrará una imagen de la ventana:

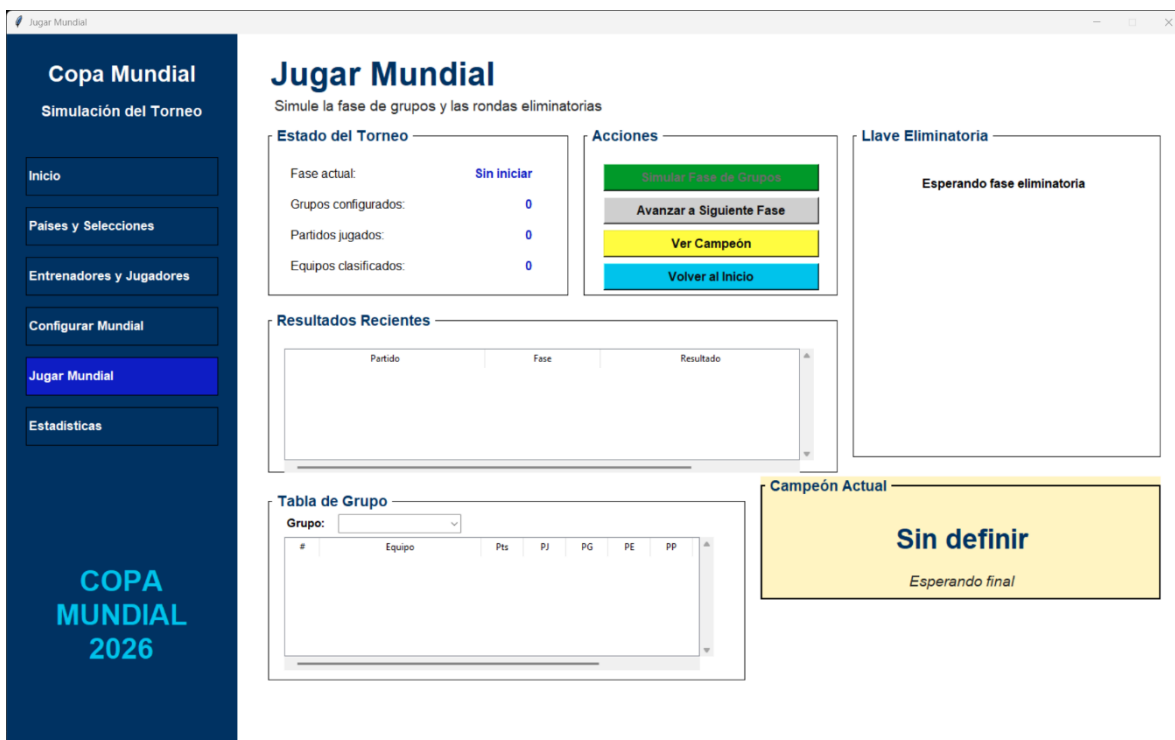
The screenshot shows a web application window titled "Configuración del Mundial". The main heading is "Configurar Mundial". Below it, a subtitle reads "Cree grupos con selecciones que tengan entrenador y de 11 a 23 jugadores". In the top right corner, there is a button labeled "Regresar al menú principal".

On the left side, under the subheading "Configuración", there is a form with a label "Cantidad de grupos:" and a dropdown menu currently set to "12". Below this, it says "Selecciones válidas: 0" and "Cada grupo usa 4 selecciones.". At the bottom of this section are three buttons: "Crear Grupos" (green), "Actualizar Datos" (blue), and "Ir a Jugar Mundial" (light blue).

On the right side, under the subheading "Grupos Formados", there is a large table with the following headers: "Grupo", "Seleccion", "Entrenador", "Jugadores", and "Fuerza". The table body is currently empty.

En la parte izquierda de la ventana se muestra la opción para asignar la cantidad de grupos que van a participar en el mundial. Cuando se selecciona la cantidad se debe presionar el botón “Crear Grupos”, y al hacerlo, se mostrarán los grupos con los países que conformarán cada grupo en el lado derecho de la ventana. Si se desea cambiar la cantidad de grupos, solo se modifica la cantidad de grupos registrados y se presiona el botón actualizar datos. Al haber tomado las decisiones necesarias, si se presiona el botón “Ir a Jugar Mundial” se abrirá la ventana de la ejecución del mundial.

4: “Jugar Mundial”: Al presionar ese botón se abrirá la ventana llamada “Jugar Mundial”, en la cual se podrá simular el desarrollo completo de la Copa Mundial, desde la fase de grupos hasta la final. En esta ventana se muestran el estado actual del torneo, los resultados de los partidos, las tablas de posiciones de cada grupo, el avance de la fase eliminatoria y el campeón del torneo una vez finalizado. A continuación, se muestra una imagen de la ventana:



al lado izquierdo de la ventana se muestra un menú de navegación, el cual muestra los botones para dirigirse a las ventanas mencionadas y la ventana de las estadísticas si se desea realizar algún cambio antes de la simulación del mundial.

En la parte superior central se encuentra la sección “Estado de torneo” donde se muestra la información del mundial si aun no ha iniciado o si ya inicio, además, a lo largo de la simulación, se muestra la fase en la que se encuentra el mundial, la cantidad de grupos que fueron creados, los partidos jugados y la cantidad de equipos que van clasificando.

Un poco a la derecha de la sección de “Estado del Torneo” se muestra la sección “Acciones” en la cual contiene los botones necesarios para la completa simulación del torneo. El botón “Simulas Fase” ejecuta los partidos que corresponden a la fase en la que se encuentre el torneo y actualiza la tabla de clasificación de los grupos, el botón “Avanzar a Siguiente Fase” se debe presionar al finalizar la simulación de la fase de grupos, la función del botón es permitir ir avanzando las rondas de eliminatorias como octavos de final, cuartos de final, semifinales y la final hasta que se obtenga el campeón del torneo. El botón “Ver Campeón” muestra la selección campeona una vez terminado el torneo por completo, y el botón “Volver al Inicio” vuelve a la ventana principal del programa.

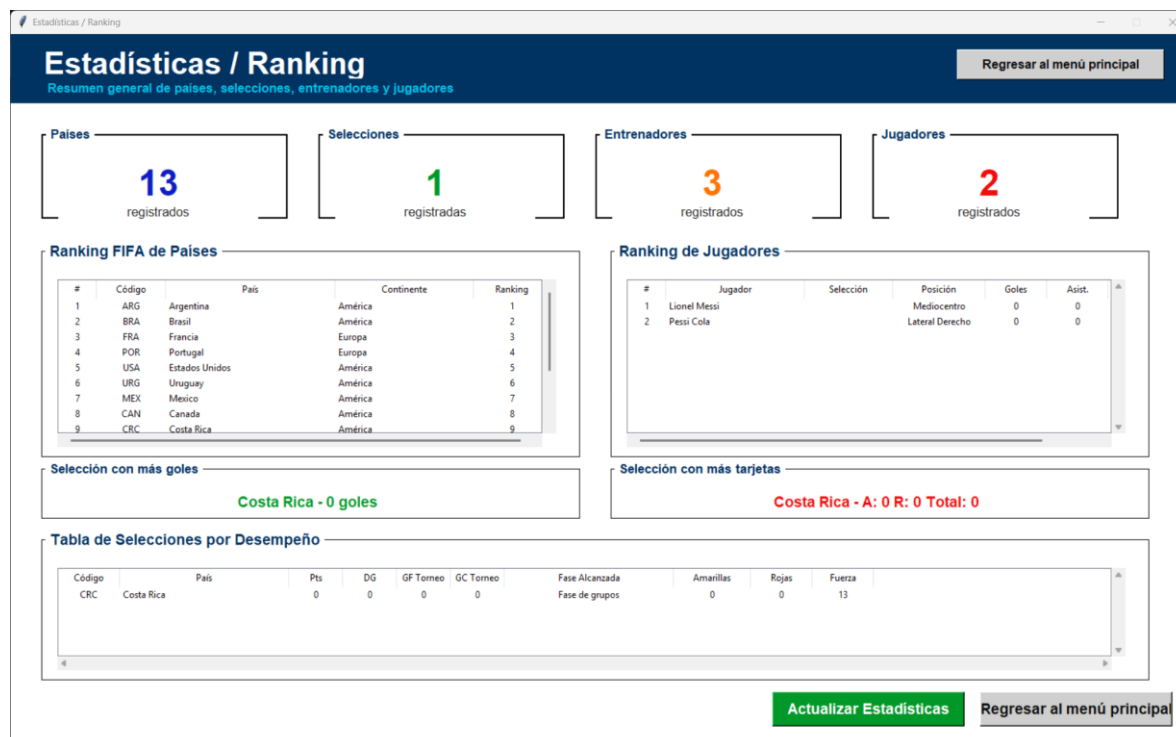
Debajo de las secciones “Estado del Torneo” y “Acciones” se encuentra el apartado “Resultados Recientes”, donde se muestran los partidos que han sido simulados junto con la fase del torneo a la que pertenecen y el resultado obtenido en cada partido. Esta información se actualiza automáticamente conforme avanza la simulación.

En la parte inferior de la ventana se muestra la sección de tabla de grupo, en la cual al seleccionar un grupo se muestran el equipo, los puntos, los partidos jugados, los partidos ganados, los partidos perdidos, los partidos empatados, los goles a favor, los goles en contra y los

En la parte derecha de la ventana se muestra el apartado “Llave Eliminatoria” en la cual se muestran los avances de las selecciones clasificadas durante las rondas de eliminación directa, esta sección se actualiza conforme avanzan las rondas hasta llegar a la final.

Finalmente, en la parte inferior derecha se muestra un apartado llamado “Campeón Actual” que es la parte en donde se mostrará la selección campeona una vez finalizado el torneo.

5: “Estadísticas/Ranking”: Al presionar ese botón se abrirá la ventana llamada “Estadísticas / Ranking”, en la cual se muestran diferentes estadísticas del torneo y de todos los datos registrados durante la ejecución del programa. En esta ventana se pueden consultar resúmenes de países, selecciones, entrenadores y jugadores, así como diferentes rankings y estadísticas generales del Mundial. A continuación, se muestra una imagen de la ventana:



Descripción del problema.

Al presentarse el mundial del año 2026 surgieron las necesidades de registrar una gran cantidad de información relacionada con este, en el mundial participan 48 selecciones, de las cuales son repartidas en grupos de 4 selecciones por cada uno, además cada selección puede contar hasta con 23 jugadores y su entrenador. Por lo que es necesario la administración de la información de manera organizada y contar con la disposición de mostrar la información almacenada de la forma mas eficiente. También hay quienes les gustaría conocer de diferentes resultados en los enfrentamientos entre países asignados aleatoriamente en diferentes grupos y simular un mundial. Por estas razones surge la necesidad del desarrollo de un programa el cual pueda simular el mundial con los países, selecciones, jugadores y entrenadores que el usuario desee.

Diseño del programa: decisiones de desarrollo, algoritmos usados.

```
def contar(elementos):  
    cantidad = 0  
  
    for elemento in elementos:  
        cantidad = cantidad + 1  
  
    return cantidad
```

La función anterior se encarga de contar los elementos de una lista o de una cadena de caracteres u otro tipo de datos que se pueda contar.


```
def numero_a_texto(numero):
    resultado = ""

    if numero == 0:
        resultado = "0"

    while numero > 0:
        digito = numero % 10

        if digito == 0:
            resultado = "0" + resultado
        elif digito == 1:
            resultado = "1" + resultado
        elif digito == 2:
            resultado = "2" + resultado
        elif digito == 3:
            resultado = "3" + resultado
        elif digito == 4:
            resultado = "4" + resultado
        elif digito == 5:
            resultado = "5" + resultado
        elif digito == 6:
            resultado = "6" + resultado
        elif digito == 7:
            resultado = "7" + resultado
        elif digito == 8:
            resultado = "8" + resultado
        elif digito == 9:
            resultado = "9" + resultado

        numero = numero // 10

    return resultado
```

la función anterior se encarga de transformar datos numéricos a datos tipo str o cadena de texto para evitar la conversión de estos.

```
def texto_es_entero(texto):
    if texto == "":
        return False

    for caracter in texto:
        if caracter != "0" and caracter != "1" and caracter != "2" and caracter != "3" and caracter != "4"
        and caracter != "5" and caracter != "6" and caracter != "7" and caracter != "8" and caracter != "9":
            return False

    return True
```

La función anterior se encarga de validar si el dato ingresado contiene solo elementos numéricos.

```
def texto_es_nombre(texto):
    if not isinstance(texto, str):
        return False

    if texto == "":
        return False

    letras = "abcdefghijklmnñopqrstuvwxyzABCDEFGHIJKLMNÑOPQRSTUVWXYZáéíóúÁÉÍÓÚ"
    tiene_letra = False

    for caracter in texto:
        if caracter in letras:
            tiene_letra = True
        elif caracter == " " or caracter == "-" or caracter == "'":
            tiene_letra = tiene_letra
        else:
            return False

    if tiene_letra == False:
        return False

    return True
```

La función anterior se encarga de validar si el dato ingresado contiene solo caracteres tipo str que sean letras, si encuentra números retorna False.

```
def texto_es_codigo(texto):
    if not isinstance(texto, str):
        return False

    if texto == "":
        return False

    letras = "abcdefghijklmnñopqrstuvwxyzABCDEFGHIJKLMNÑOPQRSTUVWXYZ"

    for caracter in texto:
        if caracter not in letras:
            return False

    return True
```

La función anterior se encarga de validar si el código FIFA ingresado contiene solo letras, si no retorna False

```

class Pais:

    def __init__(self, codigo_fifa, nombre, continente, ranking_fifa):

        if not (isinstance(codigo_fifa, str) and isinstance(nombre, str) and isinstance(continente, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not isinstance(ranking_fifa, int):
            print("Error: el parámetro debe ser int")
            return
        elif not 1 <= ranking_fifa <= 100:
            print("Error: el ranking fifa debe estar entre 1 y 100")
            return
        elif nombre == "" or continente == "" or codigo_fifa == "":
            print("Error: los parámetros no pueden estar vacíos")

        self.codigo_fifa = codigo_fifa
        self.nombre = nombre
        self.continente = continente
        self.ranking_fifa = ranking_fifa

    def mostrar_datos(self):
        print("Código FIFA del País: " + self.codigo_fifa)
        print("Nombre del País: " + self.nombre)
        print("Continente: " + self.continente)
        print("Ranking FIFA: " + numero_a_texto(self.ranking_fifa))
        print("")

    def actualizar_datos(self, codigo_fifa, nombre, continente, ranking_fifa):

        if not (isinstance(codigo_fifa, str) and isinstance(nombre, str) and isinstance(continente, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not isinstance(ranking_fifa, int):
            print("Error: el parámetro debe ser int")
            return

        self.codigo_fifa = codigo_fifa
        self.nombre = nombre
        self.continente = continente
        self.ranking_fifa = ranking_fifa

```

Clase Pais

La imagen anterior muestra la clase país, la cual será la encargada de crear los objetos de tipo país. La clase cuenta con los atributos: `codigo_fifa`, `nombre`, `continente` y `ranking_fifa`.

Además, cuenta con los siguientes métodos:

- El constructor: que crea el objeto de la clase.
- `Mostra_datos`; al ser llamado este método de la clase se muestran los datos del país.
- `Actualizar_datos`; al llamar este método de la clase se pueden modificar los datos que almacenas los atributos de la clase.

```

class Persona:
    def __init__(self, nombre, apellido, fecha_nacimiento, nacionalidad):
        if not (isinstance(nombre, str) and isinstance(apellido, str) and isinstance(fecha_nacimiento, str) and isinstance(nacionalidad, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not texto_es_nombre(nombre) or not texto_es_nombre(apellido):
            print("Error: nombre y apellido solo deben contener letras")

        self.nombre = nombre
        self.apellido = apellido
        self.fecha_nacimiento = fecha_nacimiento
        self.nacionalidad = nacionalidad

    def mostrar_datos(self):
        print("Nombre: " + self.nombre + " " + self.apellido)
        print("Fecha de Nacimiento: " + self.fecha_nacimiento)
        print("Nacionalidad: " + self.nacionalidad)
        print("")

```

Clase Persona

En la imagen anterior se muestra la clase persona, la cual es una clase padre que podrá ser heredada por las clases entrenador y jugador, esta se encarga de crear objetos de tipo persona. La clase cuenta con los siguientes atributos: nombre, apellido, fecha_nacimiento y nacionalidad.

Además, cuenta con los siguientes métodos:

- El constructor: crea el objeto relacionado con la clase.
- Mostrar_datos: muestra los datos de la persona.

```

class Entrenador(Persona):
    def __init__(self, nombre, apellido, fecha_nacimiento, nacionalidad, licencia, experiencia_anios, sistema_juego):
        Persona.__init__(self, nombre, apellido, fecha_nacimiento, nacionalidad)

        if not (isinstance(licencia, str) and isinstance(sistema_juego, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not isinstance(experiencia_anios, int):
            print("Error: el parámetro debe ser int")
            return

        self.licencia = licencia
        self.experiencia_anios = experiencia_anios
        self.sistema_juego = sistema_juego
        self.seleccion = ""

    def mostrar_datos(self):
        print("Nombre: " + self.nombre + " " + self.apellido)
        print("Fecha de Nacimiento: " + self.fecha_nacimiento)
        print("Nacionalidad: " + self.nacionalidad)
        print("Licencia: " + self.licencia)
        print("Experiencia: " + numero_a_texto(self.experiencia_anios))
        print("Sistema de Juego: " + self.sistema_juego)
        print("")

    def actualizar_datos(self, nombre, apellido, fecha_nacimiento, nacionalidad, licencia, experiencia_anios, sistema_juego):
        if not (isinstance(nombre, str) and isinstance(apellido, str) and isinstance(fecha_nacimiento, str) and isinstance(nacionalidad, str)):
            print("Error: los parámetros deben ser str")
            return
        if not (isinstance(licencia, str) and isinstance(sistema_juego, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not isinstance(experiencia_anios, int):
            print("Error: el parámetro debe ser int")
            return

        self.nombre = nombre
        self.apellido = apellido
        self.fecha_nacimiento = fecha_nacimiento
        self.nacionalidad = nacionalidad
        self.licencia = licencia
        self.experiencia_anios = experiencia_anios
        self.sistema_juego = sistema_juego

```

Clase Entrenador

La imagen anterior muestra la clase Entrenador, la cual es una clase hija de la clase Persona, esta clase es la encargada de crear los objetos de tipo entrenador. Además, la clase cuenta con los siguientes atributos: nombre, apellido, fecha_nacimiento, nacionalidad, licencia, experiencia_anios y sistema_juego. También cuenta con los siguientes métodos:

- Constructor: es el encargado de crear los objetos de tipo entrenador.
- Mostrar_datos: muestra los datos almacenados en los atributos del entrenador.
- Actualizar_datos: modifica los datos almacenados en los atributos del entrenador.

```
class Futbolista(Persona):
    def __init__(self, nombre, apellido, fecha_nacimiento, nacionalidad, dorsal, posicion, total_tarjetas_amarillas, total_tarjetas_rojas, goles, asistencias, puntaje_individual):
        Persona.__init__(self, nombre, apellido, fecha_nacimiento, nacionalidad)

        if not isinstance(posicion, str):
            print("Error: el parámetro debe ser str")
            return
        elif not (isinstance(dorsal, int) and isinstance(total_tarjetas_amarillas, int) and isinstance(total_tarjetas_rojas, int)
                  and isinstance(goles, int) and isinstance(asistencias, int) and isinstance(puntaje_individual, int)):
            print("Error: los parámetros deben ser int")
            return

        self.dorsal = dorsal
        self.posicion = posicion
        self.total_tarjetas_amarillas = total_tarjetas_amarillas
        self.total_tarjetas_rojas = total_tarjetas_rojas
        self.goles = goles
        self.asistencias = asistencias
        self.puntaje_individual = puntaje_individual
        self.seleccion = ""

    def mostrar_datos(self):
        print("Nombre: " + self.nombre + " " + self.apellido)
        print("Fecha de Nacimiento: " + self.fecha_nacimiento)
        print("Nacionalidad: " + self.nacionalidad)
        print("Dorsal: " + numero_a_texto(self.dorsal))
        print("Posicion: " + self.posicion)
        print("Tarjetas Amarillas: " + numero_a_texto(self.total_tarjetas_amarillas))
        print("Tarjetas Rojas: " + numero_a_texto(self.total_tarjetas_rojas))
        print("Goles: " + numero_a_texto(self.goles))
        print("Asistencias: " + numero_a_texto(self.asistencias))
        print("Puntaje Individual: " + numero_a_texto(self.puntaje_individual))
        print("")
```

Clase Futbolista

La imagen anterior muestra una parte de la clase Futbolista, la cual es una clase hija de la clase Persona, esta clase es la encargada de crear los objetos de tipo jugador. La clase cuenta con los siguientes atributos: nombre, apellido, fecha_nacimiento, nacionalidad, dorsal, posicion, velocidad, estrategia, dominio_balon, fuerza, total_tarjetas_amarillas, total_tarjetas_rojas, goles, asistencias y puntaje_individual.

Además, cuenta con los siguientes métodos:

- Constructor: se encarga de crear los objetos de tipo futbolista.
- mostrar_datos: muestra los datos almacenados en los atributos del jugador.
- actualizar_datos: modifica los datos almacenados en los atributos del futbolista.
- registrar_gol: suma 1 al atributo goles por cada llamada al método.
- registrar_asistencia: suma 1 al atributo asistencias por cada llamada al método.

- registrar_tarjeta: suma 1 al atributo total_tarjetas_amarillas si el dato ingresado es "amarilla", o suma 1 al atributo total_tarjetas_rojas si el dato ingresado es "roja".
- calcular_puntaje: calcula y actualiza el atributo puntaje_individual tomando en cuenta las estadísticas y el desempeño del jugador.

```
class Seleccion:

    def __init__(self, codigo_equipo, pais, entrenador):
        if not isinstance(codigo_equipo, str):
            print("Error: el parámetro debe ser str")
            return

        self.codigo_equipo = codigo_equipo
        self.pais = pais
        self.entrenador = entrenador
        self.jugadores = []
        self.total_goles_a_favor = 0
        self.total_goles_en_contra = 0
        self.total_tarjetas_amarillas = 0
        self.total_tarjetas_rojas = 0
        self.goles_torneo_favor = 0
        self.goles_torneo_contra = 0
        self.tarjetas_amarillas_torneo = 0
        self.tarjetas_rojas_torneo = 0
        self.fuerza_equipo = 0
        self.puntos = 0
        self.partidos_jugados = 0
        self.partidos_ganados = 0
        self.partidos_empatados = 0
        self.partidos_perdidos = 0
        self.diferencia_goles = 0
        self.fase_alcanzada = "Fase de grupos"

    def mostrar_datos(self):
        print("Código de la Selección: " + self.codigo_equipo)
        print("País: " + self.pais.nombre)
        if self.entrenador is not None and self.entrenador != "":
            print("Entrenador: " + self.entrenador.nombre + " " + self.entrenador.apellido)
        print("Jugadores:")
        for futbolista in self.jugadores:
            print(futbolista.nombre + " " + futbolista.apellido + " Dorsal: " + numero_a_texto(futbolista.dorsal))
        print("Goles a Favor: " + numero_a_texto(self.total_goles_a_favor))
        print("Goles en Contra: " + numero_a_texto(self.total_goles_en_contra))
        print("Fuerza de equipo: " + numero_a_texto(self.fuerza_equipo))
        print("")
```

Clase Selección

La imagen anterior muestra la clase Selección, la cual es la encargada de crear los objetos de tipo selección. La clase cuenta con los siguientes atributos: codigo, pais, entrenador, jugadores, grupo, puntos, goles_favor, goles_contra, diferencia_goles, partidos_jugados, partidos_ganados, partidos_empatados, partidos_perdidos, tarjetas_amarillas, tarjetas_rojas, fase_alcanzada y fuerza.

Además, cuenta con los siguientes métodos:

- El constructor: que crea el objeto de la clase.
- Mostrar_datos: muestra los datos de la selección.
- Agregar_jugador: agrega un futbolista a la selección.
- Eliminar_jugador: elimina un futbolista de la selección.

- Asignar_entrenador: asigna un entrenador a la selección.
- Reiniciar_estadisticas: reinicia las estadísticas de la selección y de sus jugadores.
- Registrar_estadisticas_torneo: registra las estadísticas acumuladas del torneo.
- Registrar_resultado: registra el resultado obtenido en un partido.
- Ordenar_jugadores_por_puntaje: ordena los jugadores según su puntaje individual.
- Promedio_11_mejores: calcula el promedio de puntaje de los once mejores jugadores.
- Calcular_fuerza_equipo: calcula la fuerza de la selección.

```
class Partido:

    def __init__(self, id_partido, equipo_1, equipo_2, goles_equipo1, goles_equipo2, fase, fecha):
        if not (isinstance(fase, str) and isinstance(fecha, str)):
            print("Error: los parámetros deben ser str")
            return
        elif not (isinstance(goles_equipo1, int) and isinstance(goles_equipo2, int)):
            print("Error: los parámetros deben ser int")
            return

        self.id = id_partido
        self.equipo_1 = equipo_1
        self.equipo_2 = equipo_2
        self.goles_equipo1 = goles_equipo1
        self.goles_equipo2 = goles_equipo2
        self.fase = fase
        self.fecha = fecha
        self.penales_equipo1 = 0
        self.penales_equipo2 = 0
        self.jugado = False

    def buscar_jugador_indice(self, jugadores, indice_buscado):
        indice = 0
        for jugador in jugadores:
            if indice == indice_buscado:
                return jugador
            indice += 1
        return None
```

Clase Partido

La imagen anterior muestra la clase Partido, la cual es la encargada de crear los objetos de tipo partido. La clase cuenta con los siguientes atributos: seleccion_local, seleccion_visitante, goles_local, goles_visitante, fase, estado y ganador.

Además, cuenta con los siguientes métodos:

- El constructor: que crea el objeto de la clase.
- Buscar_jugador_indice: busca un jugador según su posición en la lista.
- Registrar_goles_jugadores: registra los goles y asistencias de los jugadores.
- Calcular_tarjetas_equipo: calcula las tarjetas obtenidas por una selección.
- Registrar_tarjetas_jugadores: registra tarjetas a jugadores de una selección.
- Simular: simula el desarrollo completo del partido.
- Generar_ganador: determina el ganador del partido.
- Simular_penales: simula una tanda de penales.

- Ganador_eliminatoria: determina el ganador en una fase eliminatoria.
- Mostrar_resultado: muestra el resultado final del partido.

```
class Grupo:

    def __init__(self, nombre_grupo, equipos, partidos):
        if not isinstance(nombre_grupo, str):
            print("Error: el parámetro debe ser str")
            return
        self.nombre_grupo = nombre_grupo
        self.equipos = equipos
        self.partidos = partidos

    def agregar_equipo(self, seleccion):
        if contar(self.equipos) < 4:
            self.equipos += [seleccion]
        else:
            return "Error: el grupo ya tiene 4 selecciones"

    """
    Nombre: jugar_partidos
    Descripción: Simula los partidos que corresponden dentro del grupo
    Entrada: Selecciones guardadas en el grupo
    Salida: Partidos creados con sus resultados
    Restricción: El grupo debe tener equipos suficientes
    """
    def jugar_partidos(self):
        cantidad = contar(self.equipos)
        i = 0
        while i < cantidad - 1:
            j = i + 1
            while j < cantidad:
                partido = Partido(contar(self.partidos) + 1, self.equipos[i], self.equipos[j], 0, 0, self.nombre_grupo, "Sin fecha")
                partido.simular()
                self.partidos += [partido]
                j += 1
            i += 1

    def seleccion_va_antes(self, seleccion1, seleccion2):
        if seleccion1.puntos != seleccion2.puntos:
            return seleccion1.puntos > seleccion2.puntos
        if seleccion1.diferencia_goles != seleccion2.diferencia_goles:
            return seleccion1.diferencia_goles > seleccion2.diferencia_goles
        return seleccion1.total_goles_a_favor > seleccion2.total_goles_a_favor
```

Clase Grupo

La imagen anterior muestra la clase Grupo, la cual será la encargada de crear los objetos de tipo grupo. La clase cuenta con los atributos: nombre_grupo, equipos y partidos.

Además, cuenta con los siguientes métodos:

- El constructor: que crea el objeto de la clase.
- Agregar_equipo: agrega una selección al grupo.
- Jugar_partidos: simula todos los partidos del grupo.
- Seleccion_va_antes: compara dos selecciones para ordenarlas.
- Calcular_tabla: calcula la tabla de posiciones del grupo.
- Obtener_clasificados: obtiene las selecciones clasificadas.
- Mostrar_tabla: muestra la tabla de posiciones del grupo.


```

class Fase:

    def __init__(self, nombre_fase, partidos):
        if not isinstance(nombre_fase, str):
            print("Error: el parámetro debe ser str")
            return
        self.nombre_fase = nombre_fase
        self.partidos = partidos
        self.clasificados = []

    def registrar_juego(self, equipo1, equipo2):
        self.partidos += [Partido(contar(self.partidos) + 1, equipo1, equipo2, 0, 0, self.nombre_fase, "Sin fecha")]

    def jugar_fase(self):
        self.clasificados = []
        for partido in self.partidos:
            partido.simular()
            ganador = partido.ganador_eliminatoria()
            ganador.fase_alcanzada = self.nombre_fase
            self.clasificados += [ganador]

    def mostrar_juegos(self):
        for partido in self.partidos:
            print(partido.mostrar_resultado())

    def obtener_clasificados(self):
        return self.clasificados

```

La imagen anterior muestra la clase Fase, la cual será la encargada de crear los objetos de tipo fase eliminatoria. La clase cuenta con los atributos: nombre_fase, partidos y clasificados.

Además, cuenta con los siguientes métodos:

- El constructor: que crea el objeto de la clase.
- Registrar_juego: registra un partido en la fase.
- Jugar_fase: simula todos los partidos de la fase.
- Mostrar_juegos: muestra los resultados de los partidos.
- Obtener_clasificados: devuelve las selecciones clasificadas a la siguiente fase.
-

```

class Mundial:

    def __init__(self, nombre, anio, paises, selecciones, grupos, fases, campeon):
        if not isinstance(nombre, str):
            print("Error: el parámetro debe ser str")
            return
        elif not isinstance(anio, int):
            print("Error: el parámetro debe ser int")
            return
        self.nombre = nombre
        self.anio = anio
        self.paises = paises
        self.selecciones = selecciones
        self.grupos = grupos
        self.fases = fases
        self.campeon = campeon
        self.clasificados_actuales = []
        self.fase_actual = "Sin iniciar"
        self.partidos_jugados = []

    def registrar_pais(self, pais):
        self.paises += [pais]

    def registrar_seleccion(self, seleccion):
        self.selecciones += [seleccion]

    def seleccion_es_valida(self, seleccion):
        cantidad_jugadores = contar(seleccion.jugadores)
        return seleccion.entrenador is not None and seleccion.entrenador != "" and cantidad_jugadores >= 11 and cantidad_jugadores <= 23

```

Clase Mundial

La imagen anterior muestra la clase **Mundial**, la cual será la encargada de crear los objetos de tipo mundial. La clase cuenta con los atributos: **nombre, anio, paises, selecciones, grupos, fases, campeon, clasificados_actuales, fase_actual y partidos_jugados**.

Además, cuenta con los siguientes métodos:

- **El constructor:** que crea el objeto de la clase.
- **Registrar_pais:** registra un país en el mundial.
- **Registrar_seleccion:** registra una selección en el mundial.
- **Seleccion_es_valida:** verifica si una selección cumple los requisitos.
- **Obtener_selecciones_validas:** obtiene las selecciones válidas.
- **Reiniciar_torneo:** reinicia toda la información del torneo.
- **Nombre_grupo_por_indice:** obtiene el nombre de un grupo según su índice.
- **Crear_grupos:** crea los grupos del mundial.
- **Jugar_fase_grupos:** simula la fase de grupos.
- **Obtener_tercero_grupo:** obtiene el tercer lugar de un grupo.
- **Ordenar_terceros:** ordena los terceros lugares.
- **Obtener_mejores_terceros:** obtiene los mejores terceros.
- **Obtener_clasificados_grupos:** obtiene los clasificados de la fase de grupos.
- **Nombre_fase_por_cantidad:** determina el nombre de la siguiente fase.
- **Cantidad_es_valida_eliminatoria:** verifica que la cantidad de clasificados sea válida.
- **Armar_fase_eliminatoria:** crea la fase eliminatoria.
- **Jugar_siguiente_fase:** simula la siguiente fase eliminatoria.
- **Jugar_fase_eliminatoria:** ejecuta la fase eliminatoria.
- **Determinar_campeon:** determina el campeón del mundial.
- **Mostrar_tabla_general:** muestra las tablas de todos los grupos.
- **Generar_reporte:** genera los archivos de reporte del torneo.
- **Guardar_ranking_selecciones:** guarda el ranking de selecciones.
- **Jugador_va_antes_reporte:** compara jugadores para el ranking.
- **Ordenar_jugadores_reporte:** ordena los jugadores para el reporte.
- **Guardar_ranking_goleadores:** guarda el ranking de goleadores.

- **Texto_ganador_partido:** obtiene el nombre del ganador de un partido.
- **Guardar_partidos:** guarda el historial de partidos del torneo.

```
def leer_archivo(nombre):
    try:
        archivo = open(nombre, "r", encoding="utf-8")
        contenido = archivo.readlines()
        archivo.close()
        return contenido
    except UnicodeDecodeError:
        archivo = open(nombre, "r", encoding="cp1252")
        contenido = archivo.readlines()
        archivo.close()
        return contenido
    except:
        archivo = open(nombre, "w", encoding="utf-8")
        archivo.close()
        return []

|
contenido_paises = leer_archivo("paises.txt")
contenido_selecciones = leer_archivo("selecciones.txt")
contenido_entrenadores = leer_archivo("entrenadores.txt")
contenido_jugadores = leer_archivo("jugadores.txt")

for linea in contenido_paises:
    if linea.strip() != "":
        datos = linea.strip().split(";")
        if contar(datos) >= 4:
            lista_paises += [Pais(datos[0], datos[1], datos[2], int(datos[3]))]
```

La imagen anterior muestra la función para leer los archivos

```
def ordenar_lista_paises_por_ranking(lista):
    lista_ordenada = []

    for pais in lista:
        lista_ordenada += [pais]

    cantidad = contar(lista_ordenada)
    i = 0

    while i < cantidad - 1:
        j = i + 1

        while j < cantidad:
            if lista_ordenada[j].ranking_fifa < lista_ordenada[i].ranking_fifa:
                temporal = lista_ordenada[i]
                lista_ordenada[i] = lista_ordenada[j]
                lista_ordenada[j] = temporal

            j = j + 1

        i = i + 1

    return lista_ordenada
```

la imagen anterior muestra la función de ordenar los países por ranking fifa

```
def guardar_paises_archivo():
    archivo = open("paises.txt", "w", encoding="utf-8")
    primera_linea = True

    for pais in lista_paises:
        linea = pais.codigo_fifa + ";" + pais.nombre + ";" + pais.continente + ";" + numero_a_texto(pais.ranking_fifa)

        if primera_linea:
            archivo.write(linea)
            primera_linea = False
        else:
            archivo.write("\n" + linea)

    archivo.close()
```

La imagen anterior muestra la función para escribir los archivos

```
def cargar_entrenadores_archivo():
    global lista_entrenadores
    for linea in contenido_entrenadores:
        if linea.strip() != "":
            datos = linea.strip().split(";")
            if contar(datos) == 7 or contar(datos) == 8:
                nuevo = Entrenador(datos[0], datos[1], datos[3], datos[2], datos[4], int(datos[5]), datos[6])
                if contar(datos) == 8:
                    nuevo.seleccion = datos[7]
                lista_entrenadores += [nuevo]
```

La imagen anterior muestra como se crea la lista de los entrenadores

```
def cargar_jugadores_archivo():
    global lista_jugadores
    for linea in contenido_jugadores:
        if linea.strip() != "":
            datos = linea.strip().split(";")
            if contar(datos) == 11 or contar(datos) == 12:
                nuevo = Futbolista(datos[0], datos[1], datos[2], datos[3], int(datos[4]), datos[5], int(datos[6]), int(datos[7]), int(datos[8]), int(datos[9]), int(datos[10]))
                if contar(datos) == 12:
                    nuevo.seleccion = datos[11]
                lista_jugadores += [nuevo]
```

La imagen anterior muestra como se crea la lista de los jugadores

```
def cargar_selecciones_archivo():
    global lista_selecciones
    for linea in contenido_selecciones:
        if linea.strip() != "":
            datos = linea.strip().split(";")
            if contar(datos) == 3 or contar(datos) == 4:
                fuerza = ""
                if contar(datos) == 4:
                    fuerza = datos[3]
                nueva = crear_seleccion_desde_datos(datos[0], datos[1], datos[2], fuerza)
                if nueva is not None:
                    lista_selecciones += [nueva]
```

la imagen anterior muestra como se crea la lista de las selecciones

```
class Pantalla_Principal(tk.Tk):

    def __init__(self):
        tk.Tk.__init__(self)

        self.geometry("1535x930+-7+-0")
        self.title("Ventana")
        self.resizable(False, False)
```

La imagen anterior muestra como se crea una ventana

```
self.boton_guardar = tk.Button(self,
                                text="💾 Guardar Cambios",
                                font=("Arial", 14),
                                bd=1,
                                relief="groove",
                                fg=blanco,
                                bg=azul,
                                anchor="w",
                                command=self.actualizar,
                                state="disabled")
self.boton_guardar.place(x=480, y=350, width=170, height=40)
```

La imagen anterior muestra como se crea un botón

```

label_titulo1 = tk.Label(self,
                        text="Administración",
                        font=("Arial", 17, "bold"),
                        bg=azul_oscuro,
                        fg=blanco)
label_titulo1.place(x=30, y=10, width=200, height=30)

```

La imagen anterior muestra como se crean los labels

```

frame_principa11 = tk.Frame(self,
                            bd=1,
                            relief="solid")
frame_principa11.place(x=250, y=80, width=600, height=800)

```

La imagen anterior muestra como se crean los Frames

```

self.entry_codigo = tk.Entry(self,
                            fg=gris,
                            insertwidth=1,
                            bd=1,
                            highlightcolor=azul,
                            highlightthickness=1)
self.entry_codigo.insert(0, "Ej: CRC")
self.entry_codigo.place(x=500, y=170, width=200, height=30)

```

la imagen anterior muestra como se crean los entry

```

self.combobox_continente = ttk.Combobox(self,
                                       values=continentes,
                                       state="readonly",
                                       textvariable=self.seleccion)
self.combobox_continente.set("Seleccione un Continente")
self.combobox_continente.place(x=500, y=250, width=300, height=30)

```

La imagen anterior muestra cómo se crean los combobox

```

self.spinbox_ranking = tk.Spinbox(self,
                                from_=1,
                                to=100,
                                width=300)

self.spinbox_ranking.place(x=500, y=290, width=300, height=30)
self.spinbox_ranking.delete(0, "end")

```

la imagen anterior muestra como se crean los spinbox

```

self.tree_view = ttk.Treeview(self.tree_frame, yscrollcommand=tree_scroll.set, selectmode="extended")
self.tree_view.pack()

tree_scroll.config(command=self.tree_view.yview)

self.tree_view["columns"] = ("codigo", "nombre", "continente", "ranking")

self.tree_view.column("#0", width=0, stretch=False)
self.tree_view.column("codigo", anchor="center", width=80)
self.tree_view.column("nombre", anchor="w", width=180)
self.tree_view.column("continente", anchor="w", width=160)
self.tree_view.column("ranking", anchor="center", width=100)

self.tree_view.heading("#0", text="", anchor="w")
self.tree_view.heading("codigo", text="Código FIFA", anchor="center")
self.tree_view.heading("nombre", text="Nombre", anchor="center")
self.tree_view.heading("continente", text="Continente", anchor="center")
self.tree_view.heading("ranking", text="Ranking fifa", anchor="center")

self.tree_view.tag_configure("oddrow", background=blanco)
self.tree_view.tag_configure("evenrow", background=celeste)

```

la imagen anterior muestra un poco como se crean los treeview

```

if __name__ == "__main__":
    app = Pantalla_Principal()
    app.mainloop()

```

La imagen anterior muestra como se inicializa la ventana principal

Librerías usadas: creación de archivos, etc.

```
# Librerías

import tkinter as tk
from tkinter import ttk, messagebox
from PIL import ImageTk, Image
import random
```

Anteriormente, en la imagen se muestran las librerías que fueron utilizadas a lo largo del programa. Principalmente se utilizó la librería Tkinter para el desarrollo de la interfaz gráfica del programa, además se utilizó la librería Pillow para hacer posible mostrar imágenes que no fueran necesariamente de formato jpg. Y se utilizó la librería Random para seleccionar aleatoriamente datos que se necesitaban para la funcionalidad del proyecto.

Además, se utilizaron los siguientes archivos para guardar la información en general:

países.txt

selecciones.txt

entrenadores.txt

jugadores.txt

estadísticas.tx

Análisis de resultados:

Se lograron completar todas las instrucciones establecidas en el documento para la realización del proyecto de manera muy eficiente, tanto la registración de los países,

selecciones, entrenadores y jugadores y la simulación del mundial funciona de manera eficiente.

Conclusión:

En conclusión, este proyecto nos ayudó de gran manera en el aprendizaje del manejo de clases, sus atributos y sus métodos, también nos benefició en aprender sobre el uso de librerías como Tkinter y Random y cómo incluirlas en proyectos funcionales como este. Además de que el proyecto contiene un tema llamativo, eso nos motivó a la realización de este en todo momento.